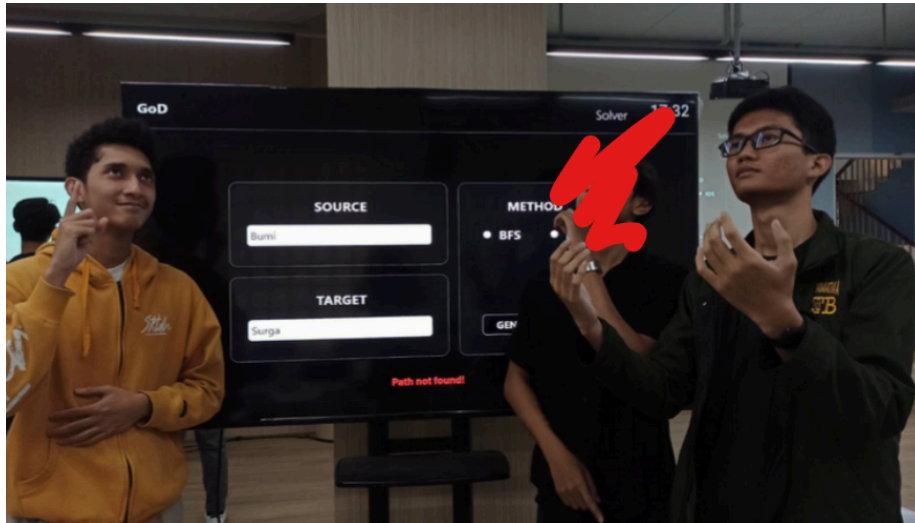


Laporan Tugas Kecil 2 (Tucil 2) IF4020 Kriptografi
Steganografi pada Berkas Audio dengan Metode Multiple-LSB
Semester I Tahun 2025/2026



Dosen Pengampu:
Dr. Ir. Rinaldi Munir, M.T.

Disusun oleh:

- | | |
|------------------------|----------|
| 1. Ahmad Rafi Maliki | 13522137 |
| 2. Muhammad Dzaki Arta | 13522149 |

INSTITUT TEKNOLOGI BANDUNG
2025

Daftar Isi

Daftar Isi.....	2	
Daftar Tabel.....	3	
Daftar Gambar.....	3	
Daftar Lampiran.....	3	
Bagian 1		
Teori Singkat.....	4	
1.1. Steganografi.....	4	
1.2. Metode Modifikasi LSB.....	4	
1.3. Berkas MP3.....	5	
1.4. Vigenère Cipher.....	6	
Bagian 2		
Perancangan dan Implementasi.....	8	
2.1. Teknologi Yang Digunakan.....	8	
2.2. Alur Kerja Program.....	8	
2.2.1. Mode Embed (Penyisipan Pesan Rahasia).....	8	
2.2.2. Mode Extract (Pengambilan Pesan Rahasia).....	9	
2.3. Proses Penyisipan.....	9	
2.4. Proses Ekstraksi.....	11	
2.5. Proses Enkripsi.....	12	
2.6. Proses Dekripsi.....	13	
Bagian 3		
Pengujian Program dan Analisis Hasil.....	15	
3.1. Pengujian.....	15	
3.2. Analisis.....	17	
Bagian 4 Kesimpulan dan Hasil Implementasi.....		18
3.1. Kesimpulan.....	18	
3.2. Hasil Implementasi.....	18	
Daftar Pustaka.....	19	
Lampiran.....	20	

Daftar Tabel

Tabel 3.1.1 Pengujian 1.....	15
Tabel 3.1.2 Pengujian 2.....	15
Tabel 3.1.3 Pengujian 3.....	15
Tabel 3.1.4 Pengujian 4.....	15
Tabel 3.1.5 Pengujian 5.....	16
Tabel 3.1.6 Pengujian 6.....	16
Tabel 3.1.7 Pengujian 7.....	16

Daftar Gambar

Gambar 1.1.1 Alur kerja steganografi.....	4
Gambar 1.2.1 Modifikasi LSB.....	5
Gambar 1.3.1 Struktur berkas mp3.....	6

Daftar Lampiran

Pranala Repositori.....	20
Pranala Video Demo.....	20
Pembagian Tugas.....	20
Tangkapan Layar.....	20

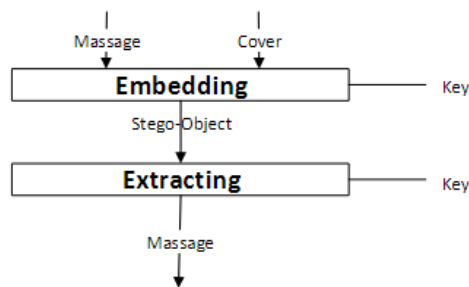
Bagian 1

Teori Singkat

Bagian ini membahas teori-teori dasar yang menjadi landasan dalam pengerjaan tugas kecil. Penjelasan mencakup konsep-konsep umum yang relevan dengan topik yaitu steganografi, mulai dari definisi, prinsip kerja, hingga teknik yang digunakan.

1.1. Steganografi

Steganografi adalah praktik menyembunyikan informasi di dalam pesan atau objek lain agar tidak mudah terdeteksi. Teknik ini dapat digunakan untuk menyisipkan berbagai jenis data digital, seperti teks, gambar, video, maupun audio. Berbeda dengan enkripsi yang hanya mengamankan isi pesan, steganografi berfokus pada penyamaran keberadaan pesan itu sendiri. Informasi yang disembunyikan kemudian dapat diekstraksi kembali secara utuh pada saat diperlukan.



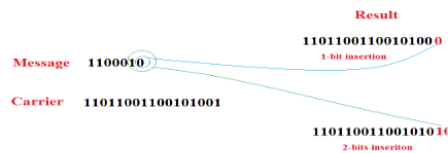
Gambar 1.1.1 Alur kerja steganografi

Dalam praktiknya, proses penyisipan pada steganografi (*embedding*) biasanya dilakukan dengan cara mengubah beberapa bit pada file asli secara sangat kecil sehingga perubahan visual atau audionya tidak terlihat oleh manusia. Contoh umum adalah memodifikasi bit-bit paling tidak signifikan (*least significant bits*) pada data gambar atau audio. Perubahan ini mengubah nilai numerik piksel atau sampel suara hanya sedikit, sehingga mata atau telinga manusia tidak menyadarinya. Selain itu, steganografi juga dapat memanfaatkan area kosong dalam struktur file, bidang metadata yang tidak dipakai, atau bagian header yang longgar untuk menyimpan data tersembunyi tanpa mengganggu fungsi atau tampilan file aslinya.

1.2. Metode Modifikasi LSB

Metode yang digunakan pada pengerjaan tugas ini adalah modifikasi *Least Significant Bit* (LSB). Pada kasus berkas audio MP3, metode ini memanfaatkan kelemahan bahwa manusia tidak begitu sensitif terhadap perubahan suara yang sangat

kecil. Dengan demikian, secara teori data yang disisipkan tidak dapat dikenali oleh indera pendengaran manusia.



Gambar 1.2.1 Modifikasi LSB

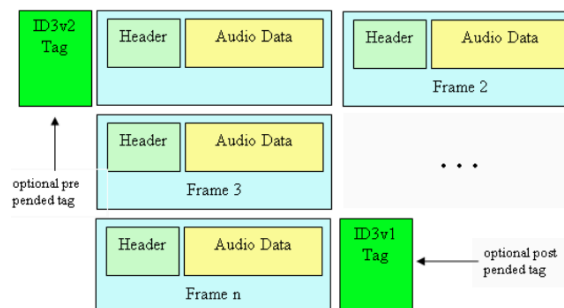
Proses penyisipan (*embedding*) dilakukan dengan cara mengganti n bit terakhir pada setiap byte data audio dengan bit-bit dari file rahasia yang ingin disisipkan. Sebaliknya, proses ekstraksi dilakukan dengan membaca kembali n bit terakhir dari byte-byte audio yang sudah dimodifikasi, kemudian menyusunnya kembali menjadi file asli.

Untuk meningkatkan keamanan, proses penyisipan dapat dilakukan secara acak dengan memanfaatkan sebuah *stego key* sebagai kunci. *Stego key* ini berfungsi sebagai umpan untuk menentukan posisi byte atau bit mana yang akan dimodifikasi, sehingga tidak hanya dilakukan secara berurutan. Dengan cara ini, pihak ketiga yang mencoba melakukan steganalisis akan semakin sulit mendeteksi pola penyisipan data.

Selain itu, demi menambah lapisan keamanan, bit-bit dari file rahasia dapat dienkripsi terlebih dahulu menggunakan algoritma enkripsi tertentu. Dengan demikian, meskipun data tersembunyi berhasil ditemukan, isinya tetap tidak dapat dibaca tanpa kunci enkripsi yang sesuai.

1.3. Berkas MP3

Agar proses penyisipan data berjalan baik, penting memahami terlebih dahulu struktur berkas MP3. Hal ini menjadi tantangan karena MP3 adalah format terkompresi, perubahan kecil pada data terkompresi dapat berpengaruh besar terhadap hasil dekompresi audio.



Gambar 1.3.1 Struktur berkas mp3

Secara umum, file MP3 tersusun dari tiga komponen utama, yaitu tag ID3v2 pada awal berkas (opsional), rangkaian *audio frames*, dan tag ID3v1 di akhir berkas (opsional). Tag ID3 (termasuk ID3v2 yang merupakan penambahan pada metode kompresi modern) menyimpan metadata seperti judul, artis, album, tahun, genre, nomor track, gambar sampul (APIC), dan komentar, serta beberapa informasi teknis yang membantu pemutaran.

Yang paling penting untuk tujuan steganografi adalah struktur *audio frame*. Data audio pada MP3 dibagi ke dalam banyak frame. Setiap frame diawali dengan header 4-byte yang dimulai oleh *sync bits*, yaitu sebuah deret bit '1' sebanyak 11 bit yang menandai awal frame. Header ini berisi informasi penting seperti versi MPEG, *layer*, indeks bitrate, frekuensi *sampling*, *flag padding*, dan mode kanal.

Setelah header biasanya ada *side information* dan kemudian *main data* (yang berisi *granule*, *scale factors*, dan data Huffman terkompresi). Pada proses penyisipan, kita harus menentukan letak setiap frame dan lokasi tepat di mana data audio terkompresi (*main data*) berada. Byte-byte inilah yang dapat dimodifikasi dengan hati-hati pada tahap embedding. Perlu ditekankan bahwa karena MP3 menggunakan kompresi kompleks, perubahan yang tampak kecil pada byte terkompresi bisa menghasilkan distorsi besar setelah dekompresi, oleh karena itu pemilihan posisi dan jumlah bit yang diubah harus dilakukan secara teliti untuk menjaga kualitas audio.

1.4. Vigenère Cipher

Metode enkripsi yang digunakan pada kode ini merupakan adaptasi dari Vigenère cipher ke dalam ruang byte (0–255). Data rahasia awalnya berbentuk deretan bit string, kemudian diubah menjadi satuan byte (8 bit per byte). Jika panjang bit terakhir kurang dari 8, maka akan dilengkapi dengan padding berupa bit '0'. Setelah terbentuk array byte, setiap byte plaintext akan dienkripsi dengan menambahkan nilai byte dari kunci (*key*) secara berulang sesuai posisinya. Proses penjumlahan dilakukan dalam ruang modulo 256 sehingga hasilnya selalu berada dalam rentang 0–255. Dengan demikian, setiap byte plaintext P_i diubah menjadi ciphertext C_i dengan rumus:

$$C_i = (P_i + K_{i(mod n)}) (mod 256)$$

Hasil enkripsi kemudian disusun kembali menjadi deretan bit (8 bit untuk setiap byte) sehingga menghasilkan string bit terenkripsi. Proses dekripsi dilakukan dengan cara kebalikan, yaitu mengurangkan byte ciphertext dengan byte kunci menggunakan operasi modulo 256:

$$P_i = (C_i - K_{i \pmod n} + 256) \pmod{256}$$

Dengan metode ini, data biner dalam bentuk bit dapat dienkripsi menggunakan kunci teks biasa, dan kunci akan diulang secara siklik jika panjang data lebih besar dari panjang kunci.

Bagian 2

Perancangan dan Implementasi

2.1. Teknologi Yang Digunakan

Program steganografi ini menggunakan command line interface (CLI) sebagai media interaksi dengan pengguna. Program ditulis menggunakan bahasa pemrograman Python versi 3.12.2, dengan memanfaatkan pustaka **tkinter** dan **questionary** untuk mendukung interaksi dengan pengguna. Dalam implementasinya, tidak digunakan pustaka khusus untuk pemrosesan file MP3 maupun jenis file lainnya.

2.2. Alur Kerja Program

Alur kerja program steganografi ini dibagi menjadi dua mode utama, yaitu ***embed*** (penyisipan pesan) dan ***extract*** (pengambilan pesan). Pemilihan mode dilakukan di awal melalui input dari pengguna.

2.2.1. Mode ***Embed*** (Penyisipan Pesan Rahasia)

Pada mode ini, program akan menjalankan tahapan berikut:

- 1) Program meminta pengguna memilih *cover file* (file media yang akan dijadikan wadah) menggunakan *file dialog* dari tkinter.
- 2) Selanjutnya, pengguna diminta memilih *secret file* (file yang akan disembunyikan).
- 3) Program menanyakan apakah pesan rahasia tersebut ingin dienkripsi.
- 4) Program menanyakan apakah penyisipan bit dilakukan menggunakan metode *random insert*.
- 5) Program meminta pengguna menentukan jumlah bit LSB (Least Significant Bit) yang akan digunakan, dengan pilihan 1 hingga 4 bit.
- 6) Jika dibutuhkan (untuk enkripsi atau *random insert*), program akan meminta kunci dari pengguna.
- 7) Program kemudian menjalankan proses penyisipan (*embedding*) sesuai parameter yang telah ditentukan.
- 8) Hasil penyisipan kemudian disimpan oleh program, dengan lokasi penyimpanan yang dipilih melalui *file dialog*.
- 9) Setelah file hasil berhasil disimpan, proses embed selesai.

2.2.2. Mode *Extract* (Pengambilan Pesan Rahasia)

Sedangkan pada mode ekstraksi, alurnya sebagai berikut:

- 1) Program meminta pengguna memilih *stega file* yang akan diekstrak menggunakan *file dialog* dari tkinter..
- 2) Program selalu menanyakan kunci, yang dapat digunakan atau diabaikan tergantung kebutuhan.
- 3) Program kemudian menjalankan proses ekstraksi (*extraction*).
- 4) Hasil ekstraksi kemudian disimpan oleh program, dengan lokasi penyimpanan yang dipilih melalui *file dialog*.
- 5) Setelah file hasil berhasil disimpan, proses ekstrak selesai.

2.3. Proses Penyisipan

Proses penyisipan dilakukan dengan menggunakan metode LSB sederhana pada *audio frame*, berikut merupakan langkahnya:

- 1) Data disisipkan pada bagian *audio frame* file mp3 yang bukan merupakan *header frame*.
- 2) Berdasarkan struktur file .mp3 yang terdiri dari ID3v2 tag (opsional), kemudian rangkaian *audio frame*, dan diakhiri oleh ID3v1 tag (opsional). Program akan menentukan terlebih dahulu letak awal (*byte offset*) untuk setiap *audio frame* yang ditandai oleh sebelas bit '1' berurutan (*sync bits*). Frame yang diperoleh juga diverifikasi terlebih dahulu dengan melakukan pemeriksaan pada bit yang diduga adalah *frame header* yang berukuran 4 byte.
- 3) Program kemudian menentukan byte mana saja yang boleh disisipi yaitu seluruh byte pada *audio frame* kecuali 4 byte awal yang merupakan *frame header*. Byte tersebut dicari *offsetnya* dan simpan ke dalam sebuah *list* yang akan kita sebut sebagai *modifiable_bytes_offset*.
- 4) Data pada *modifiable_bytes_offset* kemudian dipartisi yaitu 80 byte pertama menjadi *header_bytes_offset* dan sisanya menjadi *secret_bytes_offset*.
- 5) Jika penyisipan acak bernilai *true*, *secret_bytes_offset* yang memiliki struktur *list* akan diacak menggunakan kunci sebagai umpannya. Metode pengacakan yang digunakan adalah Fisher-Yates Shuffle.

```
def shuffle(offsets, key):
```

```

shuffled = offsets[:]
for i in range(len(shuffled) - 1, 0, -1):
    digest = hashlib.sha256(f"{key}-{i}".encode()).digest()
    j = int.from_bytes(digest, "big") % (i + 1)
    shuffled[i], shuffled[j] = shuffled[j], shuffled[i]

return shuffled

```

- 6) Jika enkripsi bernilai *true*, berkas file rahasia akan dienkripsi terlebih dahulu menggunakan metode *full vigenere cipher*.
- 7) Selanjutnya akan disusun *header* untuk file rahasia sebesar 10 byte, dengan format, 4 byte pertama merepresentasikan ukuran file rahasia dalam byte, bit ke-lima merepresentasikan *n lsb* yang digunakan, bit ke-enam menyimpan *flag* enkripsi dan penyisipan acak, dan 4 bit terakhir menyimpan informasi ekstensi file rahasia.

```

def create_secret_header(num_secret_bytes, n_lsb, is_encrypted,
is_random_insertion, ext):

    header = bytearray(10)
    header[0:4] = num_secret_bytes.to_bytes(4, 'big')
    header[4] = n_lsb
    flags = 0
    if is_encrypted:
        flags |= 0b00000001
    if is_random_insertion:
        flags |= 0b00000010
    header[5] = flags
    header[6:10] = ext.encode().ljust(4, b'\0')

    return header

```

- 8) *Header* kemudian disisipkan pada alamat *header_bytes_offset* dengan menggunakan *n lsb* bernilai 1.

```

def write_header_bits(cover_bytes, offsets, header_bits):

    for i in range(0, 80):
        bit_to_embed = header_bits[i:i+1]
        bit_to_embed = bit_to_embed.ljust(1, '0')

        lsb_bits = f'{cover_bytes[offsets[i]]:08b}'[-1:]
        if lsb_bits != bit_to_embed:
            cover_bytes[offsets[i]] &= (0xFF << 1)
            cover_bytes[offsets[i]] |= int(bit_to_embed, 2)

    return cover_bytes

```

- 9) File rahasia kemudian akan dikonversi menjadi bit dan bit tersebut akan disisipkan pada alamat *secret_bytes_offset* dengan menggunakan *n lsb* sesuai masukan pengguna.

```
def write_secret_bits(cover_bytes, offsets, secret_bits, n_lsb):  
    for i in range(0, len(secret_bits), n_lsb):  
        bit_to_embed = secret_bits[i:i+n_lsb]  
        bit_to_embed = bit_to_embed.ljust(n_lsb, '0')  
  
        index = (i // n_lsb) + 80  
  
        lsb_bits = f'{cover_bytes[offsets[index]]:08b}'[-n_lsb:]  
  
        if lsb_bits != bit_to_embed:  
            cover_bytes[offsets[index]] &= (0xFF << n_lsb)  
            cover_bytes[offsets[index]] |= int(bit_to_embed, 2)  
  
    return cover_bytes
```

- 10) Setelah seluruh bit rahasia disisipkan, proses *embedding* selesai.

2.4. Proses Ekstraksi

Proses penyisipan dilakukan dengan menggunakan metode LSB sederhana pada *audio frame*, berikut merupakan langkahnya:

- 1) Program memperoleh *header_bytes_offset* dan juga *secret_bytes_offset*, sama seperti yang sudah dijelaskan pada proses *embedding*.
- 2) Program akan melakukan parsing terhadap *header* dari file rahasia untuk memperoleh data ukuran file, *n lsb* yang digunakan, apakah file dienkrpsi, apakah penyisipan dilakukan secara acak, dan apa format file rahasianya.

```
def read_header_bits(stego_bytes, offsets):  
    header_bits = ''  
    num_bits = 10 * 8  
    for i in range(num_bits):  
        byte_offset = offsets[i]  
        byte_value = stego_bytes[byte_offset]  
        lsb_bits = f'{byte_value:08b}'[-1:]  
        header_bits += lsb_bits  
  
    return header_bits
```

```
def parse_secret_header(header_bits):  
  
    if len(header_bits) != 80:  
        raise ValueError("Invalid header length in bits")
```

```

header_bytes = bit_to_byte(header_bits)

num_secret_bytes = int.from_bytes(header_bytes[0:4], 'big')
n_lsb = header_bytes[4]
flags = header_bytes[5]
is_encrypted = (flags & 0b00000001) != 0
is_random_insertion = (flags & 0b00000010) != 0
ext = header_bytes[6:10].rstrip(b'\0').decode()

return {
    "num_secret_bytes": num_secret_bytes,
    "n_lsb": n_lsb,
    "is_encrypted": is_encrypted,
    "is_random_insertion": is_random_insertion,
    "ext": ext
}

```

- 3) Jika penyisipan dilakukan secara acak, maka *secret_bytes_offset* akan diacak menggunakan kunci sebagai umpan.
- 4) Program kemudian akan mengambil *n lsb* dari byte-byte pada *secret_bytes_offset*, dan menyusunnya menjadi satu kesatuan bit stream.

```

def read_secret_bits(stego_bytes, offsets, total_secret_bits,
n_lsb):

    secret_bits = ''
    start_index = 80

    bytes_needed = (total_secret_bits + n_lsb - 1) // n_lsb

    for i in range(bytes_needed):
        byte_offset = offsets[start_index + i]
        byte_value = stego_bytes[byte_offset]
        lsb_bits = f'{byte_value:08b}'[-n_lsb:]
        secret_bits += lsb_bits

    return secret_bits[:total_secret_bits]

```

- 5) Jika *flag* enkripsi bernilai *true*, maka bitstream yang diperoleh akan didekripsi menggunakan kunci.
- 6) Setelah seluruh bit disusun dan proses dekripsi dilakukan, proses *extraction* selesai.

2.5. Proses Enkripsi

Proses enkripsi dilakukan dengan menggunakan metode Extended Vigenère Cipher. Adapun langkah-langkah enkripsinya adalah sebagai berikut

- 1) Berkas rahasia dibaca sebagai sebuah urutan byte (disebut sebagai *plaintext*).

- 2) Kunci yang dimasukkan oleh pengguna juga dikonversi menjadi urutan byte.
- 3) Setiap byte dari *plaintext* dienkripsi dengan menambahkan nilainya dengan nilai byte dari kunci. Kunci akan diulang jika panjangnya lebih pendek dari panjang *plaintext*. Proses ini dapat direpresentasikan dengan rumus:

$$C_i = (P_i + K_{i(mod\ n)}) (mod\ 256)$$

Dimana:

- C_i adalah byte hasil enkripsi (ciphertext) pada posisi ke-i.
 - P_i adalah byte plaintext pada posisi ke-i.
 - $K_{i(mod\ n)}$ adalah byte kunci pada posisi ke-i (dengan pengulangan), di mana n adalah panjang kunci.
- 4) Hasil dari proses ini adalah urutan byte baru yang telah terenkripsi, yang kemudian akan dikonversi menjadi bit untuk disisipkan ke dalam berkas audio.

Adapun kode untuk enkripsi pesan adalah sebagai berikut

```
def encrypt_bits(secret_file_bits: str, key: str) -> str:
    key_bytes = key.encode("utf-8")
    key_len = len(key_bytes)

    secret_bytes = bytearray()
    for i in range(0, len(secret_file_bits), 8):
        bit_chunk = secret_file_bits[i:i+8]
        if len(bit_chunk) < 8:
            bit_chunk = bit_chunk.ljust(8, '0')
        secret_bytes.append(int(bit_chunk, 2))

    encrypted = bytearray()
    for i, byte in enumerate(secret_bytes):
        k = key_bytes[i % key_len]
        encrypted.append((byte + k) % 256)

    encrypted_bits = ''.join(format(b, '08b') for b in encrypted)
    return encrypted_bits
```

2.6. Proses Dekripsi

Proses dekripsi dilakukan setelah bit-bit pesan rahasia berhasil diekstrak dari berkas audio (stego file). Proses ini hanya akan dijalankan jika flag enkripsi yang tersimpan di dalam header bernilai true. Metode yang digunakan adalah invers dari Extended Vigenère Cipher, dengan menggunakan kunci yang sama seperti saat enkripsi. Langkah-langkah dekripsinya adalah sebagai berikut:

- 1) Bitstream hasil ekstraksi disusun kembali menjadi sebuah urutan byte (ciphertext).
- 2) Kunci yang dimasukkan oleh pengguna saat proses ekstraksi digunakan untuk proses dekripsi.
- 3) Setiap byte dari ciphertext didekripsi dengan mengurangi nilainya dengan nilai byte dari kunci. Sama seperti enkripsi, kunci akan diulang jika diperlukan. Proses ini direpresentasikan oleh rumus:

$$P_i = (C_i - K_{i(mod n)} + 256) (mod 256)$$

Dimana:

- C_i adalah byte ciphertext pada posisi ke-i.
 - P_i adalah byte hasil dekripsi (plaintext) pada posisi ke-i.
 - $K_{i(mod n)}$ adalah byte kunci yang sesuai.
 - Penambahan 256 dilakukan untuk memastikan hasil operasi modulo tidak negatif.
- 4) Hasil dari proses ini adalah urutan byte asli dari berkas rahasia sebelum didekripsi.

Adapun kode untuk proses dekripsi pesan adalah sebagai berikut:

```
def decrypt_bits(secret_bits: str, key: str) -> str:
    key_bytes = key.encode("utf-8")
    key_len = len(key_bytes)

    secret_bytes = bytearray()
    for i in range(0, len(secret_bits), 8):
        bit_chunk = secret_bits[i:i+8]
        if len(bit_chunk) < 8:
            bit_chunk = bit_chunk.ljust(8, '0')
        secret_bytes.append(int(bit_chunk, 2))

    decrypted = bytearray()
    for i, byte in enumerate(secret_bytes):
        k = key_bytes[i % key_len]
        decrypted.append((byte - k + 256) % 256)

    decrypted_bits = ''.join(format(b, '08b') for b in decrypted)
    return decrypted_bits
```

Bagian 3

Pengujian Program dan Analisis Hasil

3.1. Pengujian

Bagian ini mendeskripsikan seluruh kasus pengujian yang dilakukan pada program steganografi. Seluruh berkas yang digunakan pada proses pengujian, disimpan pada repositori GitHub.

Tabel 3.1.1 Pengujian 1

Kasus uji	Tanpa enkripsi, titik sisip tidak acak, $n\ lsb = 1$
Berkas masukan (<i>cover</i>)	files/input/mp3/Queen – Bohemian Rhapsody.mp3
Berkas masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
Berkas keluaran	files/output/tc1.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip terdengar seperti robot di awal pemutaran lagu untuk beberapa detik

Tabel 3.1.2 Pengujian 2

Kasus uji	Dengan enkripsi, titik sisip tidak acak, $n\ lsb = 1$
File masukan (<i>cover</i>)	files/input/mp3/Queen – Bohemian Rhapsody.mp3
File masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
File keluaran	files/output/tc2.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip terdengar seperti robot di awal pemutaran lagu untuk beberapa detik.

Tabel 3.1.3 Pengujian 3

Kasus uji	Tanpa enkripsi, titik sisip acak, $n\ lsb = 1$
File masukan (<i>cover</i>)	files/input/mp3/maroon5 - memories.mp3
File masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
File keluaran	files/output/tc3.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip masih terdengar noise yang mengganggu saat mendengarkan tetapi lagunya masih bisa didengar.

Tabel 3.1.4 Pengujian 4

Kasus uji	Dengan enkripsi, titik sisip acak, $n\ lsb = 1$
File masukan (<i>cover</i>)	files/input/mp3/maroon5 - memories.mp3

File masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
File keluaran	files/output/tc4.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip masih terdengar noise yang mengganggu saat mendengarkan tetapi lagunya masih bisa didengar.

Tabel 3.1.5 Pengujian 5

Kasus uji	Dengan enkripsi, titik sisip acak, $n\ lsb = 2$
File masukan (cover)	files/input/mp3/maroon5 - memories.mp3
File masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
File keluaran	files/output/tc5.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip masih terdengar noise yang mengganggu saat mendengarkan tetapi lagunya masih bisa didengar. Noise terdengar lebih besar dibanding dengan pengujian 3 dan pengujian 4

Tabel 3.1.6 Pengujian 6

Kasus uji	Dengan enkripsi, titik sisip acak, $n\ lsb = 4$
File masukan (cover)	files/input/mp3/maroon5 - memories.mp3
File masukan (sisip)	files/input/txt/Kage no Jitsuryokusha ni Naritakute! – 1.txt
File keluaran	files/output/tc6.mp3
Keterangan (hasil)	Berkas mp3 hasil sisip masih terdengar noise yang mengganggu saat mendengarkan tetapi lagunya masih bisa didengar. Noise terdengar lebih besar dibanding dengan pengujian 5

Tabel 3.1.7 Pengujian 7

Kasus uji	File sisip melebihi kapasitas
File masukan (cover)	files/input/mp3/maroon5 - memories.mp3
File masukan (sisip)	files/input/txt/bigfile.txt
File keluaran	-
Keterangan (hasil)	Program akan mengeluarkan error “An error occurred: Secret file is too large to be embedded in the cover file with the given number of LSBs.”.

Tabel 3.1.8 Pengujian 8

Kasus uji	File sisip berformat docx
File masukan (cover)	files/input/mp3/Grand Escape _ A Weathering With You AMV

	[YAXTn0E-Zgo].mp3
File masukan (sisip)	files/input/docx/Proklamasi.docx
File keluaran	files/output/tc8.mp3
Keterangan (hasil)	Penyisipan dan ekstraksi file docx berhasil dilakukan

3.2. Analisis

Pada proses pengujian, tidak ditemukan perbedaan yang signifikan terkait variasi jumlah bit LSB yang digunakan maupun proses ada tidaknya dekripsi pada penyisipan, ataupun perbedaan yang dipengaruhi oleh genre musik tertentu atau jenis file sisipnya. Perbedaan yang lebih jelas justru muncul pada metode penyisipan acak dibandingkan dengan metode tidak acak. Pada penyisipan tidak acak, bagian awal berkas audio mengalami kerusakan cukup parah hingga audio asli tidak dapat lagi dikenali oleh indera pendengaran manusia. Sebaliknya, pada penyisipan acak, audio asli masih terdengar relatif jelas meskipun terdapat noise tambahan yang dapat disadari.

Fenomena ini terjadi karena proses penyisipan dilakukan secara langsung (naif) pada data terkompresi, sehingga menimbulkan kesalahan fatal ketika file didekompresi. Akibatnya, hasil audio tidak lagi sesuai dengan data yang seharusnya. Pada metode penyisipan tidak acak, kerusakan ini lebih mudah terdeteksi karena byte data yang rusak tersusun berurutan di bagian awal audio. Sementara itu, pada penyisipan acak, kerusakan lebih tersamarkan karena byte yang rusak tersebar dengan jarak tertentu, sehingga sulit dikenali secara langsung oleh pendengaran manusia.

Hal penting yang perlu diperhatikan adalah proses *embedding* sangat bergantung pada pola persebaran bit sisipan agar file asli tidak mengalami kerusakan signifikan. Namun, konsekuensinya, ukuran data yang dapat disisipkan relatif sangat kecil dibandingkan dengan ukuran berkas audio MP3 itu sendiri.

Bagian 4 Kesimpulan dan Hasil Implementasi

3.1. Kesimpulan

Program steganografi berhasil diimplementasikan dengan memanfaatkan konsep *least significant bit*, penyisipan bit acak, serta kriptografi. Program mampu melakukan *embedding* dengan menghasilkan file audio yang tetap terjaga integritasnya, sekaligus memungkinkan *ekstraksi* kembali file rahasia secara utuh. Namun, keterbatasan yang muncul akibat mekanisme kompresi MP3 membuat ukuran file rahasia yang dapat disisipkan relatif sangat kecil dibandingkan dengan ukuran file cover.

Proses *embedding* yang lebih optimal sebenarnya dapat dilakukan untuk menghasilkan kualitas file yang lebih baik serta kapasitas penyisipan yang lebih besar. Akan tetapi, hal tersebut memerlukan pemahaman mendalam mengenai metode kompresi MP3 serta perhitungan matematis yang kompleks. Oleh karena itu, dalam konteks tugas ini, fokus diarahkan pada penerapan konsep dasar steganografi itu sendiri, tanpa memperluas lingkup pada detail teknis kompresi MP3.

3.2. Hasil Implementasi

Tangkapan layar hasil implementasi dapat dilihat pada lampiran tangkapan layar.

Daftar Pustaka

Kaspersky. (t.t.). *What is Steganography?* Kaspersky Resource Center. Diakses pada 4 Oktober 2025, dari <https://www.kaspersky.com/resource-center/definitions/what-is-steganography>

Alghamdi, A. (2023). *Audio Steganography Method Using Least Significant Bit (LSB) Encoding Technique*. ResearchGate. Diakses pada 4 Oktober 2025, dari https://www.researchgate.net/publication/369708559_Audio_Steganography_Method_Using_Least_Significant_Bit_LSB_Encoding_Technique

ITM Web of Conferences. (2022). *Adaptation for Vigenère Cipher Method for Auto Binary Files Ciphering*. Dalam *ITM Web of Conferences* (Vol. 46, p. 01017). EDP Sciences. Diakses pada 4 Oktober 2025, dari https://www.itm-conferences.org/articles/itmconf/pdf/2022/02/itmconf_icacs2022_01017.pdf

Lampiran

Pranala Repositori

https://github.com/rafimaliki/Tucil2-Kriptografi_13522137_13522149

Pranala Video Demo

<https://youtu.be/A9mSxu6onKo>

Pembagian Tugas

Tabel. Pembagian tugas

Anggota	Tugas
13522137 Ahmad Rafi Maliki	Implementasi algoritma embed dan extract LSB
13522149 Muhammad Dzaki Arta	Implementasi algoritma enkripsi dan melakukan pengujian

Tangkapan Layar

```
(venv) C:\Files\Source-Code\Kuliah\Code-Semester-7\IF4020_Kriptografi\Tucil_2\Tucil2-Kriptografi_13522137_13522149\sro [P main]> py main.py

Welcome to the MP3 Audio File Steganography Program!

/$$ /$$ /$$$$$$ /$$$$$ /$$$$$ /$$$$$$ /$$$$$ /$$$$$ /$$$$$
|$$$ /$$$ $- $$ /$- $$ /$- $$ /$- $$ /$- $$ /$- $$
$$$ /$$$ $- \ $ / \ $ / \ $ / \ $ / \ $ / \ $ / \ $
$$ /$$ $ /$$$$$$ /$$$$$ /$$$$$ /$$$$$ /$$$$$ /$$$$$ /$$$$$
$$ $$$ $ /$$$ /$$$ /$$$ /$$$ /$$$ /$$$ /$$$ /$$$ /$$$
$$ \ $ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$
$$ \ / $ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$ /$
--/ --/ --/ --/ --/ --/ --/ --/ --/ --/ --/ --/ --/ --/

? Select mode: (Use arrow keys)
> Embed
  Extract
```

Gambar 1. Memilih mode program

